

UNIT 1 — Software Engineering & Agile Methodology

PART A — 1 Mark Answers

1. Define Software

Software is a collection of programs, procedures, and related documentation that instructs a computer to perform specific tasks.

2. What is Software Engineering?

Software Engineering is a systematic and disciplined approach used for designing, developing, testing, and maintaining software systems.

3. Who is a Software Engineer?

A software engineer is a professional who designs, develops, tests, and maintains software applications using engineering principles.

4. List one characteristic of good software

Reliability. (Correctness, Reliability, Efficiency, Maintainability, User-friendliness, Security, Scalability)

5. What is Software Crisis?

Software crisis refers to the problems faced in software development such as high cost, delays, poor quality, and difficulty in maintenance.

6. What does SDLC stand for?

Software Development Life Cycle.

7. What is a Software Process?

A software process is a structured set of activities used to develop and maintain software systems.

8. What is the main idea of the Waterfall Model?

Development progresses sequentially through phases such as requirement, design, coding, testing, and maintenance.

9. What is the key focus of the Spiral Model?

Risk analysis and iterative development.

10. Define Agile Methodology

Agile is an iterative and incremental software development approach that focuses on flexibility, customer collaboration, and continuous delivery.

11. What is Extreme Programming (XP)?

Extreme Programming is an Agile method that improves software quality through practices like pair programming and test-driven development.

12. When was the Agile Manifesto introduced?

2001.

13. What is a Sprint in Scrum?

A sprint is a short development cycle (usually 2–4 weeks) in which a specific set of tasks is completed.

14. What is a self-organizing team?

A team that manages its own work and decides how to complete tasks without external control.

PART B — 2 Marks

1. Differentiate between Software and Hardware

Software	Hardware
Program instructions	Physical components
Intangible	Tangible
Easily modified	Difficult to modify

2. Two objectives of Software Engineering

- To produce high-quality software
 - To complete software within budget and time
-

3. Two characteristics of good software

- Reliability
 - Maintainability
-

4. Two causes of Software Crisis

- Increasing complexity of software
 - Poor project management
-

5. Phases of SDLC

1. Requirement Analysis
 2. Design
 3. Implementation
 4. Testing
 5. Deployment
 6. Maintenance
-

6. Software Process vs SDLC

Software Process

SDLC

Software Process

Overall development activities

Defines workflow

SDLC

Specific lifecycle phases

Defines stages

7. Incremental Model

Software is developed in small parts called increments. Each increment delivers partial functionality.

8. Prototyping Model Use

Used when requirements are unclear or not well understood.

9. Two features of Agile

- Iterative development
 - Continuous customer feedback
-

10. Role of Product Owner

Represents the customer and manages the product backlog.

PART C — 3 Marks

1. Importance of Software Engineering

Software engineering is important because it:

- Improves software quality
 - Reduces development cost
 - Helps manage complex systems
 - Ensures reliable and maintainable software
-

2. Software Crisis

Software crisis refers to problems faced during software development.

Problems include:

- Delays in project completion
- High development cost

- Poor quality software
- Difficult maintenance

Software engineering was introduced to solve these issues.

3. Phases of SDLC

1. Requirement Analysis
2. System Design
3. Implementation
4. Testing
5. Deployment
6. Maintenance

These phases ensure systematic software development.

4. Waterfall Model

The Waterfall Model is a sequential development process where each phase must be completed before the next phase begins.

Advantages

- Simple to understand
- Well structured

Disadvantages

- No flexibility
 - Difficult to change requirements
-

5. Spiral Model

The Spiral Model combines iterative development with risk analysis.

Each loop includes:

- Planning
- Risk Analysis
- Engineering
- Evaluation

It is suitable for large projects.

6. Agile Methodology

Agile is a flexible development approach where software is built in small iterations.

Benefits:

- Faster delivery
 - Better customer satisfaction
 - Easy requirement changes
-

7. Extreme Programming Practices

- Pair Programming
 - Test Driven Development
 - Continuous Integration
 - Small Releases
 - Refactoring
-

PART D — 4 Marks

1. Compare Process Models

Model	Feature
Waterfall	Sequential development
Incremental	Development in parts
Prototyping	Prototype built first
Spiral	Risk driven model
Agile	Iterative and flexible

2. SDLC with Diagram

Requirement → Design → Coding → Testing → Deployment → Maintenance

SDLC ensures systematic development of software.

3. Agile Manifesto Values

1. Individuals over processes
 2. Working software over documentation
 3. Customer collaboration over contract negotiation
 4. Responding to change over following a plan
-

4. Scrum Framework

Scrum consists of:

Roles

- Product Owner
- Scrum Master
- Development Team

Artifacts

- Product Backlog
- Sprint Backlog
- Increment

Events

- Sprint Planning
 - Daily Scrum
 - Sprint Review
 - Sprint Retrospective
-

5. Extreme Programming

XP focuses on fast development and high quality.

Advantages

- Fast development
- Continuous testing

Disadvantages

- Not suitable for large teams.

UNIT-2

Requirement Engineering & Design Concepts

PART A — 1 Mark Answers

1. What is Requirement Engineering?

Requirement Engineering is the systematic process of gathering, analyzing, documenting, validating, and managing the requirements of a software system.

2. Define Requirement Modeling.

Requirement modeling is the process of representing system requirements using diagrams and models to help understand system functionality clearly.

3. What is an SRS document?

SRS (Software Requirement Specification) is a document that describes the functional and non-functional requirements of a software system.

4. What are functional requirements?

Functional requirements describe the specific functions or services that the system must perform.

5. What are non-functional requirements?

Non-functional requirements define system quality attributes such as performance, security, reliability, and usability.

6. What is a stakeholder?

A stakeholder is any person or organization that has an interest in the software system.

7. What is requirement elicitation?

Requirement elicitation is the process of collecting requirements from stakeholders.

8. What is validation in Requirement Engineering?

Validation is the process of checking whether the requirements are correct, complete, and feasible.

9. Define Software Design.

Software design is the process of converting system requirements into a blueprint for building software.

10. What is Software Architecture?

Software architecture defines the overall structure of the system and relationships between its components.

11. What is cohesion?

Cohesion refers to how closely related the functions inside a module are.

12. What is coupling?

Coupling refers to the level of dependency between different modules.

13. What is a Use Case Diagram?

A Use Case Diagram shows the interaction between users (actors) and the system.

14. What is a Class Diagram?

A Class Diagram represents classes, attributes, methods, and relationships between classes.

15. What is a Sequence Diagram?

A Sequence Diagram shows how objects interact in a particular sequence of time.

16. What is an Activity Diagram?

An Activity Diagram represents the workflow of activities in a system.

PART B — 2 Marks Answers

1. Seven Tasks of Requirement Engineering

The requirement engineering process includes the following tasks:

1. **Inception** – Identifying stakeholders and project scope.
 2. **Requirement Elicitation** – Collecting requirements from stakeholders.
 3. **Elaboration** – Analyzing and refining requirements.
 4. **Negotiation** – Resolving conflicts between stakeholders.
 5. **Specification** – Documenting requirements in SRS.
 6. **Validation** – Checking correctness and feasibility.
 7. **Requirement Management** – Managing requirement changes.
-

2. Functional vs Non-Functional Requirements

Functional Requirements	Non-Functional Requirements
Define system behavior	Define system quality
Describe system functions	Describe system constraints
Example: Login system	Example: System response time

3. Components of an SRS Document

Main components include:

1. **Introduction** – Purpose and scope of the system.
 2. **Overall Description** – System overview and assumptions.
 3. **Functional Requirements** – System functions.
 4. **Non-Functional Requirements** – Performance, security etc.
 5. **External Interface Requirements** – Interaction with users or systems.
-

4. Requirement Validation

Requirement validation ensures that the requirements are:

- Correct
- Complete
- Consistent
- Feasible

Validation methods include:

- Reviews
- Prototyping
- Test case generation

5. Characteristics of a Good SRS

A good SRS should be:

- Correct
- Complete
- Consistent
- Unambiguous
- Verifiable

6. Design Concepts in Software Engineering

Design concepts are principles used to structure software systems effectively.

Examples:

- Abstraction
- Modularity
- Cohesion
- Coupling

7. Abstraction and Modularity

Abstraction

Abstraction hides internal details and shows only essential features.

Example: ATM machine hides internal banking processes.

Modularity

Modularity divides a system into smaller modules.

Advantages:

- Easier maintenance
- Better debugging

8. Cohesion vs Coupling

Cohesion

Relation inside module

High cohesion is good

Coupling

Relation between modules

Low coupling is good

9. Architectural Design

Architectural design defines the overall system structure and interactions between components.

Examples:

- Layered architecture
 - Client-server architecture
 - MVC architecture
-

10. Purpose of UML Diagrams

UML diagrams help to:

- Visualize system structure
 - Represent system behaviour
 - Improve communication between developers
-

PART C — 3 Marks Answers

1. Seven Tasks of Requirement Engineering

Requirement engineering is the process of identifying and managing system requirements.

The seven tasks include:

1. Inception

Identifying stakeholders and defining system objectives.

2. Requirement Elicitation

Gathering requirements using techniques like interviews, surveys, and observation.

3. Elaboration

Analyzing and refining gathered requirements.

4. Negotiation

Resolving conflicts between stakeholders.

5. Specification

Documenting requirements in the SRS document.

6. Validation

Ensuring requirements are correct and feasible.

7. Requirement Management

Handling requirement changes throughout the project.

2. Gathering Functional and Non-Functional Requirements

Functional requirements define system functions such as login, registration, and payment.

Non-functional requirements define system quality attributes such as:

- Performance
- Security
- Reliability
- Usability

Requirements are collected using:

- Interviews
 - Questionnaires
 - Workshops
 - Observation
-

3. Requirement Modeling

Requirement modeling is the representation of system requirements using diagrams.

Types of requirement models include:

1. **Scenario-based modeling** – Use Case diagrams.
2. **Class-based modeling** – Class diagrams.
3. **Behavioral modeling** – Sequence diagrams.
4. **Flow-oriented modeling** – Data Flow Diagrams.

Benefits:

- Improves understanding
 - Helps detect missing requirements
 - Enhances communication.
-

4. Structure of SRS Document

The SRS document contains:

1. Introduction

Purpose and scope of the system.

2. Overall Description

General system characteristics.

3. Functional Requirements

Detailed system functions.

4. Non-Functional Requirements

Performance, reliability, and security requirements.

5. External Interfaces

User interface, hardware interface, and software interface.

SRS serves as an agreement between the client and developers.

5. Design Principles

Important design principles include:

DRY (Don't Repeat Yourself)

Avoid duplication of code.

KISS (Keep It Simple)

Keep the design simple.

High Cohesion

Each module should perform a single task.

Low Coupling

Modules should be independent of each other.

PART D — 4 Marks

1. Requirement Engineering in Detail

Requirement engineering is the process of identifying, documenting, validating, and managing system requirements.

Steps include:

1. Inception
2. Requirement Elicitation

3. Elaboration
4. Negotiation
5. Specification
6. Validation
7. Requirement Management

Requirement engineering ensures that the developed software meets user expectations and reduces development errors.

2. Software Requirement Specification (SRS)

SRS is a formal document describing system requirements.

Structure:

1. Introduction
2. Overall Description
3. Functional Requirements
4. Non-Functional Requirements
5. Interface Requirements

Importance:

- Acts as a contract between client and developer
 - Helps in project planning
 - Reduces misunderstandings
-

3. Software Design Concepts

Important design concepts include:

Abstraction – Hiding unnecessary details.

Modularity – Dividing system into modules.

Cohesion – Degree of relation within module.

Coupling – Dependency between modules.

A good design should have **high cohesion and low coupling**.

4. Architectural Design

Architectural design defines system structure.

Types include:

Layered Architecture

Presentation Layer

Business Layer

Data Layer

Client-Server Architecture

Client sends request → Server processes → Response returned.

MVC Architecture

Model – Data

View – User interface

Controller – Business logic

5. UML Diagrams

Important UML diagrams include:

Use Case Diagram

Shows interaction between user and system.

Class Diagram

Represents system structure.

Sequence Diagram

Shows message exchange over time.

Activity Diagram

Represents system workflow.

UNIT-3 Software Quality Assurance & Maintenance

1. What is Software Quality?

Software quality refers to the degree to which software meets specified requirements and user expectations without defects.

2. Define Software Quality Assurance (SQA).

Software Quality Assurance (SQA) is a set of activities used to ensure that software processes and products meet quality standards.

3. What is Software Reliability?

Software reliability is the ability of software to perform correctly without failure for a specified period of time.

4. What is a Formal Technical Review (FTR)?

A Formal Technical Review (FTR) is a structured process of reviewing software documents, design, or code to identify errors and improve quality.

5. What is the main objective of SQA?

The main objective of SQA is to ensure high-quality software by preventing defects and improving development processes.

6. What is ISO 9000?

ISO 9000 is a set of international standards for quality management systems that help organizations maintain product and service quality.

7. What is CMM?

Capability Maturity Model (CMM) is a framework used to measure and improve software development processes.

8. What is Six Sigma?

Six Sigma is a quality improvement methodology used to reduce defects and improve process performance.

9. What does CMM stand for?

CMM stands for **Capability Maturity Model**.

10. What is the purpose of quality standards?

The purpose of quality standards is to improve software quality, ensure consistency, increase customer satisfaction, and reduce defects.

11. What is Software Maintenance?

Software maintenance is the process of modifying and updating software after delivery to correct errors, improve performance, or adapt to changes.

12. Name any one type of maintenance.

One type of maintenance is **Corrective Maintenance**. (Other types: Adaptive, Perfective, Preventive)

13. What is Reverse Engineering?

Reverse engineering is the process of analyzing existing software to understand its design, structure, and functionality.

14. What is Forward Engineering?

Forward engineering is the process of developing software from design and specifications into the final system.

15. What is Re-engineering?

Re-engineering is the process of modifying and improving existing software to enhance quality, performance, or maintainability.

1. Explain Software Quality Assurance in brief.

Software Quality Assurance (SQA)

Software Quality Assurance (SQA) is a set of planned and systematic activities used to ensure that software processes and products meet specified quality standards and requirements.

SQA focuses on preventing defects rather than only detecting them.

Objectives of SQA

- Improve software quality
- Prevent defects
- Ensure standard development process
- Increase customer satisfaction
- Improve reliability and performance

Importance of SQA

- Produces high-quality software
 - Reduces maintenance cost
 - Improves productivity
 - Ensures timely delivery
-

2. What are the activities of SQA?

The major activities of Software Quality Assurance are:

1. **Requirement Analysis**
Reviewing software requirements for correctness and completeness.
 2. **Process Monitoring**
Ensuring development follows defined standards and procedures.
 3. **Formal Technical Reviews (FTR)**
Reviewing design, code, and documents to identify defects.
 4. **Testing Activities**
Conducting various software tests to detect errors.
 5. **Audit and Inspection**
Checking whether processes follow quality standards.
 6. **Defect Tracking and Reporting**
Recording and monitoring software defects.
 7. **Configuration Management**
Managing software versions and changes.
 8. **Documentation Control**
Maintaining proper project documents and reports.
-

3. Explain Formal Technical Review (FTR).

Formal Technical Review (FTR)

A Formal Technical Review is a structured process of reviewing software documents, design, code, or requirements to identify errors and improve software quality.

It is performed by a group of technical experts.

Objectives of FTR

- Detect defects early
- Verify compliance with standards
- Improve software quality
- Reduce development cost

Types of Documents Reviewed

- Requirement documents
- Design documents
- Source code
- Test plans

Steps in FTR

1. Planning the review
2. Reviewing the material
3. Identifying defects
4. Recording review results
5. Correcting defects

Advantages

- Early error detection
 - Improves maintainability
 - Reduces testing effort
-

4. What is Software Reliability? Explain briefly.

Software Reliability

Software reliability is the probability that software will perform its required functions correctly without failure for a specified period under given conditions.

Features

- Measures software dependability
- Indicates error-free operation
- Improves customer trust

Factors Affecting Reliability

- Number of defects
- Software complexity
- Testing quality
- Operating environment

Importance

- Ensures stable software operation
- Reduces system failures
- Improves software performance

Example

A banking system operating continuously without crashes is considered reliable.

5. Difference between Quality Control and Quality Assurance.

Quality Assurance (QA)

Process-oriented

Focuses on preventing defects

Improves development process

Performed throughout development

Example: Process audit

Quality Control (QC)

Product-oriented

Focuses on detecting defects

Improves final product

Performed after development

Example: Software testing

Conclusion

- QA focuses on preventing problems.
 - QC focuses on finding and fixing problems.
-

6. Explain ISO 9000 in brief.

ISO 9000

ISO 9000 is a family of international quality management standards used to ensure consistent quality in products and services.

It provides guidelines for maintaining quality management systems.

Objectives

- Improve product quality
- Increase customer satisfaction
- Ensure process consistency

- Improve organizational efficiency

Features

- Internationally recognized standard
- Focuses on quality management process
- Applicable to all organizations

Advantages

- Better product quality
- Improved customer confidence
- Reduced operational errors

7. Explain Capability Maturity Model (CMM).

Capability Maturity Model (CMM)

Capability Maturity Model (CMM) is a framework developed to measure and improve software development processes in an organization.

It helps organizations achieve better quality and process maturity.

Objectives

- Improve software process quality
- Increase productivity
- Reduce development risks

Levels of CMM

1. Initial Level

Processes are unpredictable and unorganized.

2. Repeatable Level

Basic project management processes are established.

3. Defined Level

Processes are standardized and documented.

4. Managed Level

Processes are measured and controlled.

5. Optimizing Level

Continuous process improvement is performed.

Advantages

- Improves software quality
- Better project management
- Reduces defects and risks

8. What is Six Sigma? Explain its importance.

Six Sigma

Six Sigma is a quality management methodology used to improve processes by reducing defects and errors.

It focuses on achieving high quality and efficiency through continuous improvement.

The main goal of Six Sigma is to reduce defects to a very low level.

Features of Six Sigma

- Data-driven approach
- Focuses on customer satisfaction
- Improves process quality
- Reduces variation and defects

Six Sigma Methodology (DMAIC)

1. **Define** – Identify the problem
2. **Measure** – Measure current performance
3. **Analyze** – Find root causes of defects
4. **Improve** – Improve the process
5. **Control** – Maintain improvements

Importance of Six Sigma

- Improves software quality
 - Reduces defects and errors
 - Increases productivity
 - Saves time and cost
 - Improves customer satisfaction
 - Enhances business performance
-

9. Compare ISO and CMM.

ISO

International quality standard

Focuses on quality management

Applicable to all industries

Provides quality certification

Defines minimum quality standards

Developed by ISO organization

CMM

Process improvement model

Focuses on software process maturity

Mainly used in software industry

Provides maturity levels

Measures process capability

Developed by Software Engineering Institute (SEI)

Conclusion

- ISO ensures quality standards.
 - CMM improves software development processes.
-

10. What is SQA Plan? Explain briefly.

SQA Plan

An SQA (Software Quality Assurance) Plan is a document that describes the procedures, standards, tools, and activities used to ensure software quality during development.

It acts as a roadmap for quality assurance activities.

Objectives of SQA Plan

- Ensure software quality
- Define quality standards
- Prevent defects
- Improve development process

Components of SQA Plan

1. Quality objectives
2. Testing strategy
3. Review and audit procedures
4. Standards and methods
5. Roles and responsibilities
6. Documentation process
7. Defect reporting process

Importance of SQA Plan

- Improves project management
 - Reduces software defects
 - Ensures standard process
 - Increases customer confidence
-

11. Explain types of Software Maintenance.

Software maintenance is the process of modifying software after delivery to improve or correct it.

Types of Software Maintenance

1. Corrective Maintenance

Fixes bugs and errors found after software release.

Example

Fixing login issues in an application.

2. Adaptive Maintenance

Modifies software to work in a changed environment.

Example

Updating software for a new operating system.

3. Perfective Maintenance

Improves performance or adds new features.

Example

Adding dark mode to an application.

4. Preventive Maintenance

Improves maintainability and prevents future problems.

Example

Optimizing old source code.

12. What is Corrective and Adaptive Maintenance?

Corrective Maintenance

Corrective maintenance is performed to fix defects, bugs, or errors discovered after software deployment.

Example

Correcting calculation errors in payroll software.

Purpose

- Remove software defects
- Improve system stability

Adaptive Maintenance

Adaptive maintenance is performed to modify software according to changes in environment or technology.

Example

Updating software to support a new database version.

Purpose

- Maintain compatibility
- Adapt to new environments

Difference

Corrective Maintenance	Adaptive Maintenance
Fixes errors and bugs	Adapts software to changes
Focuses on defect correction	Focuses on environment changes
Improves correctness	Improves compatibility

13. Explain Reverse Engineering with example.

Reverse Engineering

Reverse engineering is the process of analyzing existing software to understand its design, structure, and functionality.

It converts software from implementation level to design level.

Objectives

- Understand old software
- Recover lost documentation
- Improve maintenance

Example

Analyzing an old banking application's source code to understand how it works before updating it.

Advantages

- Helps understand complex systems
- Supports software maintenance
- Assists in re-engineering

14. Explain Re-engineering.

Re-engineering

Software re-engineering is the process of modifying and improving existing software to enhance quality, performance, maintainability, or functionality.

It updates old software without completely rebuilding it.

Objectives

- Improve software quality
- Reduce maintenance cost
- Modernize software system

Activities in Re-engineering

1. Reverse Engineering
2. Code restructuring
3. Data restructuring
4. Forward Engineering

Example

Converting an old desktop banking system into a modern web-based application.

Advantages

- Extends software life
- Improves maintainability
- Saves development cost

15. Difference between Forward and Reverse Engineering.

Forward Engineering

Converts design into software

Development process

Creates new software

Starts from requirements/design

Used in software development

Reverse Engineering

Converts software into design information

Analysis process

Studies existing software

Starts from existing code

Used in maintenance and understanding

Example

- **Forward Engineering:** Developing a new mobile app from design documents.
- **Reverse Engineering:** Studying source code of an old application to understand its functionality.

1. Explain Software Quality Assurance with diagram.

Software Quality Assurance (SQA)

Software Quality Assurance (SQA) is a set of planned and systematic activities used to ensure that software processes and products meet specified quality standards and customer requirements. SQA focuses on preventing defects and improving the software development process.

Objectives of SQA

- Ensure high-quality software
- Prevent defects
- Improve development process
- Ensure compliance with standards
- Increase customer satisfaction

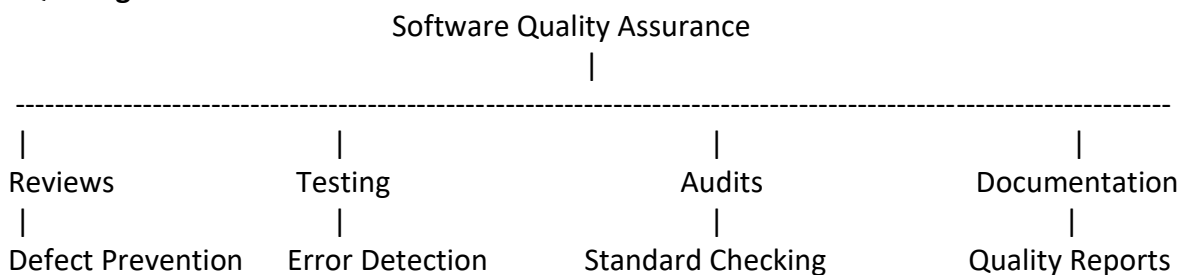
Activities of SQA

1. Requirement analysis
2. Process monitoring
3. Formal technical reviews
4. Software testing
5. Audits and inspections
6. Defect tracking
7. Documentation control

Importance of SQA

- Improves software reliability
- Reduces maintenance cost
- Detects defects early
- Improves productivity
- Ensures customer satisfaction

SQA Diagram



Conclusion

SQA helps organizations produce reliable, high-quality, and defect-free software by following proper quality standards and processes.

2. Explain Formal Technical Review (FTR) process in detail.

Formal Technical Review (FTR)

Formal Technical Review (FTR) is a structured review process used to identify defects in software documents, design, code, and other work products before testing begins.

It is performed by a team of technical experts.

Objectives of FTR

- Detect errors early
- Verify compliance with standards
- Improve software quality
- Reduce development cost
- Improve maintainability

Documents Reviewed in FTR

- Requirement specifications
 - Design documents
 - Source code
 - Test plans
-

FTR Process

1. Planning

The review leader selects the review team and schedules the review meeting.

2. Overview Preparation

The author explains the document or code to reviewers.

3. Review Meeting

Reviewers examine the document carefully and identify defects, inconsistencies, and errors.

4. Defect Recording

All detected defects are documented properly.

5. Rework

The author corrects identified defects.

6. Follow-up

The review leader verifies whether all corrections are completed successfully.

Advantages of FTR

- Early error detection
- Reduces testing effort
- Improves software quality
- Improves team communication

FTR Diagram

Planning



Preparation



Review Meeting



Defect Recording



Rework



Follow-up

Conclusion

FTR is an effective quality assurance activity that helps detect defects early and improves the overall software development process.

3. Explain Software Reliability and its importance.

Software Reliability

Software reliability is the probability that software will perform its required functions correctly without failure for a specified period under given conditions.

It measures the dependability and stability of software.

Features of Software Reliability

- Error-free operation
- Stable performance
- Consistent functionality

- Reduced system failures

Factors Affecting Reliability

1. Number of defects
2. Software complexity
3. Quality of testing
4. Operating environment
5. User workload

Importance of Software Reliability

1. Reduces Failures

Reliable software operates without frequent crashes or errors.

2. Increases Customer Satisfaction

Users trust software that works consistently.

3. Improves Safety

Reliability is critical in banking, medical, and aviation systems.

4. Reduces Maintenance Cost

Fewer failures reduce repair and maintenance expenses.

5. Ensures Business Continuity

Reliable systems avoid downtime and data loss.

Example

A banking system that performs transactions continuously without errors is considered reliable.

Reliability Diagram

Better Testing



Fewer Defects



Higher Reliability



Customer Satisfaction

Conclusion

Software reliability is an essential quality factor that ensures stable, dependable, and failure-free software performance.

4. Explain SQA Plan with components.

SQA Plan

An SQA (Software Quality Assurance) Plan is a document that defines the quality assurance activities, standards, tools, methods, and responsibilities used during software development.

It acts as a roadmap for maintaining software quality.

Objectives of SQA Plan

- Ensure software quality
- Prevent defects
- Define quality standards
- Improve development process
- Monitor quality activities

Components of SQA Plan

1. Purpose and Objectives

Defines quality goals of the project.

2. Scope

Specifies features and processes covered by SQA activities.

3. Standards and Procedures

Defines coding standards, testing methods, and review procedures.

4. Roles and Responsibilities

Defines responsibilities of developers, testers, and managers.

5. Review and Audit Activities

Specifies technical reviews and audits to be performed.

6. Testing Strategy

Defines testing methods and tools.

7. Defect Reporting Process

Defines how defects are recorded and tracked.

8. Documentation Control

Maintains project documents and reports.

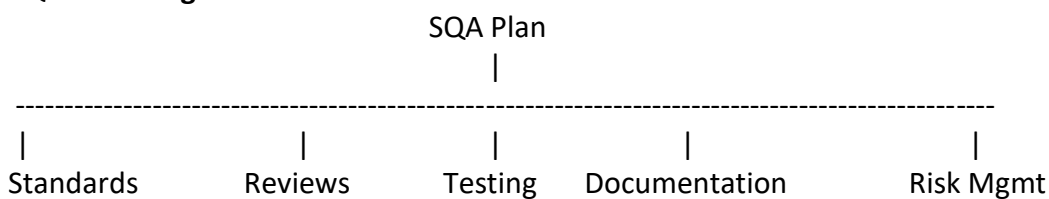
9. Risk Management

Identifies possible risks and solutions.

Importance of SQA Plan

- Improves software quality
- Reduces defects
- Ensures process consistency
- Helps project management
- Increases customer confidence

SQA Plan Diagram



5. Explain ISO 9000 standards in detail.

ISO 9000 Standards

ISO 9000 is a family of international quality management standards used to ensure that organizations consistently provide quality products and services.

These standards are developed by the International Organization for Standardization (ISO).

Objectives of ISO 9000

- Improve product and service quality
- Increase customer satisfaction
- Standardize organizational processes
- Improve efficiency and productivity
- Reduce defects and errors

Features of ISO 9000

- Internationally accepted standards
 - Focus on quality management system (QMS)
 - Continuous process improvement
 - Customer-oriented approach
 - Proper documentation and control
-

Main Principles of ISO 9000

1. Customer focus
 2. Leadership
 3. Employee involvement
 4. Process approach
 5. Continuous improvement
 6. Evidence-based decision making
-

Benefits of ISO 9000

- Better product quality
 - Improved customer trust
 - Increased efficiency
 - Reduced operational errors
 - Better project management
-

Importance in Software Industry

- Ensures standard development process
 - Improves software quality
 - Reduces software defects
 - Increases customer confidence
-

ISO 9000 Diagram

Customer Requirements



Quality Management System



Standardized Processes



Quality Product/Service



Customer Satisfaction

Conclusion

ISO 9000 helps organizations maintain quality standards, improve processes, and deliver reliable products and services.

6. Explain CMM levels with diagram.

Capability Maturity Model (CMM)

Capability Maturity Model (CMM) is a framework used to measure and improve software development processes in an organization.

It helps organizations achieve process maturity and better software quality.

Objectives of CMM

- Improve software quality
 - Standardize development process
 - Reduce defects and risks
 - Increase productivity
-

CMM Levels

1. Initial Level

- Processes are unorganized and unpredictable.
- Success depends on individual effort.

Characteristics

- No defined process
- High risk

2. Repeatable Level

- Basic project management processes are established.
- Previous success can be repeated.

Characteristics

- Requirement management
- Project planning

3. Defined Level

- Processes are documented and standardized across the organization.

Characteristics

- Standard procedures
- Training programs

4. Managed Level

- Processes are measured and controlled using metrics.

Characteristics

- Quality measurement
- Process monitoring

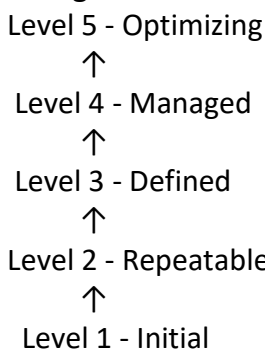
5. Optimizing Level

- Continuous improvement of processes is performed.

Characteristics

- Process innovation
- Defect prevention

CMM Diagram



Advantages of CMM

- Improves software quality
- Better project management
- Reduces defects
- Increases productivity

Conclusion

CMM helps organizations improve software processes systematically through different maturity levels.

7. Explain Six Sigma methodology (DMAIC).

Six Sigma

Six Sigma is a quality improvement methodology used to reduce defects and improve process performance.

It uses statistical and management techniques to achieve high quality.

DMAIC Methodology

DMAIC is the main process used in Six Sigma.

1. Define

Identify the problem, project goals, and customer requirements.

Example

Finding reasons for software delivery delays.

2. Measure

Collect data and measure current process performance.

Example

Measuring number of software defects per module.

3. Analyze

Analyze collected data to identify root causes of defects.

Example

Finding coding errors causing failures.

4. Improve

Implement solutions to remove defects and improve process.

Example

Introducing better coding standards and reviews.

5. Control

Monitor improvements and maintain process quality.

Example

Regular audits and performance monitoring.

DMAIC Diagram

Define



Measure



Analyze



Improve



Control

Importance of Six Sigma

- Reduces defects
- Improves software quality
- Saves cost and time
- Increases customer satisfaction
- Improves efficiency

Conclusion

Six Sigma helps organizations continuously improve quality and reduce defects using the DMAIC approach.

8. Compare ISO 9000, CMM and Six Sigma.

ISO 9000	CMM	Six Sigma
International quality standard	Process maturity model	Quality improvement methodology
Focuses on quality management system	Focuses on software process improvement	Focuses on defect reduction
Applicable to all industries	Mainly used in software industry	Used in many industries
Provides certification	Provides maturity levels	Provides process improvement techniques
Ensures standard process	Measures process capability	Improves process performance
Developed by ISO organization	Developed by SEI	Developed by Motorola

Key Focus Comparison

Standard/Model Main Focus

ISO 9000	Quality management
CMM	Process maturity
Six Sigma	Defect reduction and improvement

- ISO 9000 ensures organizations follow quality standards.
 - CMM improves software development maturity.
 - Six Sigma reduces defects and improves process efficiency.
-

9. Explain all types of Software Maintenance in detail.

Software Maintenance

Software maintenance is the process of modifying and updating software after delivery to correct errors, improve performance, or adapt to changing requirements and environments.

Types of Software Maintenance

1. Corrective Maintenance

Definition

Corrective maintenance is performed to fix defects, bugs, or errors discovered after software deployment.

Objectives

- Remove software errors
- Improve correctness and stability

Example

Fixing incorrect salary calculation in payroll software.

Advantages

- Improves reliability
 - Removes operational issues
-

2. Adaptive Maintenance

Definition

Adaptive maintenance is performed to modify software according to changes in technology, hardware, operating systems, or business environment.

Objectives

- Maintain compatibility
- Adapt software to new environment

Example

Updating software to work with a new version of Windows or database.

Advantages

- Extends software usability
 - Supports new technologies
-

3. Perfective Maintenance

Definition

Perfective maintenance is performed to improve software performance, efficiency, or add new features according to user requirements.

Objectives

- Enhance functionality
- Improve performance and usability

Example

Adding dark mode feature to an application.

Advantages

- Increases user satisfaction
 - Improves software efficiency
-

4. Preventive Maintenance

Definition

Preventive maintenance is performed to improve maintainability and prevent future problems before they occur.

Objectives

- Reduce future risks
- Improve software maintainability

Example

Refactoring old source code and improving documentation.

Advantages

- Reduces future failures
 - Simplifies maintenance activities
-

Comparison of Maintenance Types

Type	Purpose	Example
Corrective	Fix defects	Fixing login bug
Adaptive	Adapt to environment changes	Supporting new OS
Perfective	Improve features/performance	Adding new functionality
Preventive	Prevent future problems	Code optimization

Conclusion

All maintenance types help keep software reliable, efficient, and updated according to user and business needs.

10. Explain Re-engineering process with steps.

Software Re-engineering

Software re-engineering is the process of analyzing and modifying existing software to improve quality, maintainability, and performance without completely rebuilding the system. It modernizes old software systems.

Objectives of Re-engineering

- Improve software quality
- Reduce maintenance cost
- Increase maintainability
- Modernize legacy systems

Steps in Re-engineering Process

1. Inventory Analysis

Identify software applications that require re-engineering.

Example

Selecting outdated banking software for improvement.

2. Document Restructuring

Update or improve old documentation.

Example

Creating proper design documents for old software.

3. Reverse Engineering

Analyze existing software to understand its structure and functionality.

Example

Studying old source code.

4. Code Restructuring

Improve code quality and readability without changing functionality.

Example

Removing duplicate code.

5. Data Restructuring

Improve database structure and data organization.

Example

Normalizing database tables.

6. Forward Engineering

Develop improved software from the recovered design and information.

Example

Converting desktop software into web application.

Re-engineering Process Diagram

Inventory Analysis



Document Restructuring



Reverse Engineering



Code/Data Restructuring



Forward Engineering

Advantages of Re-engineering

- Extends software life
- Improves maintainability
- Reduces development cost
- Improves performance

Conclusion

Re-engineering improves old software systems and makes them more efficient, maintainable, and modern.

11. Explain Reverse Engineering and Forward Engineering in detail.

Reverse Engineering

Definition

Reverse engineering is the process of analyzing existing software to understand its design, structure, functionality, and components.

It converts implementation-level information into higher-level design information.

Objectives

- Understand old systems
 - Recover missing documentation
 - Support maintenance and re-engineering
-

Process of Reverse Engineering

1. Analyze source code
 2. Extract design information
 3. Understand system architecture
 4. Prepare documentation
-

Example

Studying source code of an old hospital management system to understand how it works.

Advantages

- Helps understand complex systems
 - Supports software maintenance
 - Assists in modernization
-

Forward Engineering

Definition

Forward engineering is the process of developing software from requirements and design specifications into the final working system.

It converts design into implementation.

Steps in Forward Engineering

1. Requirement analysis
 2. System design
 3. Coding
 4. Testing
 5. Deployment
-

Example

Developing a new mobile banking application from design documents.

Advantages

- Produces new software systems
 - Supports structured development
 - Improves software quality
-

Difference between Reverse and Forward Engineering

Reverse Engineering

Converts software into design information

Used for analysis and maintenance

Forward Engineering

Converts design into software

Used for software development

Reverse Engineering

Starts from existing code

Helps understand old systems

Forward Engineering

Starts from requirements/design

Helps build new systems

Conclusion

Reverse engineering helps understand existing systems, while forward engineering helps create new software systems.

12. Explain Software Maintenance process model.

Software Maintenance Process Model

The Software Maintenance Process Model describes the sequence of activities performed to modify and maintain software after delivery.

It ensures software continues to function correctly and efficiently.

Steps in Software Maintenance Process

1. Problem Identification

Identify defects, change requests, or improvement needs.

Example

User reports software crash issue.

2. Analysis

Analyze the problem and determine required modifications.

Example

Finding the root cause of the crash.

3. Design the Solution

Prepare design changes and modification plan.

Example

Updating module design for correction.

4. Implementation

Modify source code and apply required changes.

Example

Correcting faulty program logic.

5. Testing

Test modified software to ensure changes work correctly.

Example

Performing regression testing.

6. Acceptance

Users verify and approve the modified software.

Example

Client confirms issue is fixed.

7. Release and Documentation

Release updated software and update documentation.

Example

Providing software patch and updated manuals.

Software Maintenance Process Diagram

Problem Identification



Analysis



Solution Design



Implementation



Testing



Acceptance



Release & Documentation

Importance of Maintenance Process Model

- Improves software quality
 - Reduces future failures
 - Maintains software performance
 - Ensures customer satisfaction
-

Conclusion

The software maintenance process model provides a systematic approach for updating and improving software after deployment.

UNIT-4 Software Testing Fundamentals:

1. What is Software Testing?

Software testing is the process of checking a software application to find defects and verify that it works according to requirements.

2. Why is software testing important?

Software testing is important because it:

- Finds bugs/errors
- Improves software quality
- Ensures security and reliability
- Reduces maintenance cost
- Increases customer satisfaction

3. Define Verification.

Verification is the process of checking whether the software is being developed correctly according to specifications.

“Are we building the product right?”

4. Define Validation.

Validation is the process of checking whether the final software meets user needs and requirements.

“Are we building the right product?”

5. What is STLC?

STLC stands for **Software Testing Life Cycle**.

It is a sequence of activities performed during testing such as:

- Requirement Analysis
- Test Planning
- Test Case Development
- Environment Setup
- Test Execution
- Test Closure

6. What is Unit Testing?

Unit testing is testing individual components or modules of a program separately.

7. What is Integration Testing?

Integration testing checks whether combined modules work properly together.

8. What is System Testing?

System testing tests the complete integrated software system as a whole.

9. What is Acceptance Testing?

Acceptance testing is performed to verify whether the software satisfies business requirements and is ready for delivery.

10. What is Manual Testing?

Manual testing is testing software manually without using automation tools.

11. What is Automation Testing?

Automation testing uses tools and scripts to execute test cases automatically.

12. What is Functional Testing?

Functional testing checks whether software functions work according to requirements.

13. What is Non-Functional Testing?

Non-functional testing checks performance, security, usability, reliability, etc.

14. What is Black-box Testing?

Black-box testing tests software functionality without knowing internal code structure.

15. What is White-box Testing?

White-box testing tests internal code, logic, and program structure.

16. What is Grey-box Testing?

Grey-box testing is a combination of black-box and white-box testing where partial internal knowledge is used.

17. What is Boundary Value Analysis?

Boundary Value Analysis is a testing technique that checks values at boundaries such as minimum, maximum, just below, and just above limits.

18. What is Equivalence Partitioning?

Equivalence Partitioning divides input data into valid and invalid groups where one test case can represent each group.

19. What is a Test Case?

A test case is a set of conditions, inputs, and expected results used to test a feature of software.

20. What is a Test Plan?

A test plan is a document describing testing scope, objectives, strategy, resources, schedule, and activities.

1. Explain Software Testing and its importance.

Software Testing

Software testing is the process of checking and evaluating a software application to identify defects and ensure that it works according to user requirements.

Importance of Software Testing

- Helps find bugs and errors before delivery
- Improves software quality and reliability
- Ensures security and performance
- Reduces maintenance cost
- Increases customer satisfaction
- Ensures the software meets requirements

2. Differentiate between Verification and Validation.

Verification

Checks whether the product is developed correctly

Process-oriented

Done during development

Does not execute code

Example: Reviews, inspections

Validation

Checks whether the correct product is developed

Product-oriented

Done after development

Executes the software

Example: Testing the application

Simple Line

- **Verification:** "Are we building the product right?"
 - **Validation:** "Are we building the right product?"
-

3. Explain STLC phases briefly.

STLC stands for **Software Testing Life Cycle**.

It consists of the following phases:

1. **Requirement Analysis**
Study and analyze requirements for testing.
 2. **Test Planning**
Prepare test strategy, resources, schedule, and tools.
 3. **Test Case Development**
Create test cases and test data.
 4. **Environment Setup**
Prepare hardware and software testing environment.
 5. **Test Execution**
Execute test cases and record results.
 6. **Defect Reporting and Tracking**
Report bugs and track their fixes.
 7. **Test Closure**
Evaluate testing process and prepare final reports.
-

4. What is Qualification in software testing?

Qualification in software testing is the process of ensuring that software meets specified standards, requirements, and user expectations before release.

It verifies that:

- The software functions correctly
- Quality standards are achieved
- The system is reliable and safe to use

Types of Qualification

- Installation Qualification (IQ)
- Operational Qualification (OQ)
- Performance Qualification (PQ)

Importance

- Ensures product quality
 - Reduces failures and risks
 - Increases customer confidence
-

5. Explain Unit Testing with example.

Unit Testing

Unit testing is a type of testing in which individual components or modules of a program are tested separately to verify that each unit works correctly.

It is usually performed by developers during coding.

Advantages

- Finds bugs early
- Simplifies debugging
- Improves code quality

Example

Suppose there is a function for addition:

```
int add(int a, int b){  
    return a + b;  
}
```

Testing:

- add(2,3) → Expected Output: 5
- add(5,7) → Expected Output: 12

Here, only the add() function is tested independently, so it is unit testing.

6. Explain Integration Testing.

Integration Testing

Integration testing is the process of testing combined modules or components together to check whether they interact properly.

It is performed after unit testing.

Objectives

- Detect interface errors between modules
- Verify data flow between components
- Ensure integrated modules work correctly

Example

A banking application has:

- Login Module
- Account Module
- Payment Module

Integration testing checks whether these modules communicate correctly with each other.

Types

- Top-down Integration Testing
- Bottom-up Integration Testing

- Big Bang Integration Testing

7. Explain System Testing.

System Testing

System testing is the testing of the complete integrated software system as a whole to verify that it meets specified requirements.

It is performed after integration testing.

Features

- Tests the entire application
- Performed in a real-like environment
- Includes functional and non-functional testing

Example

Testing a complete e-commerce website including:

- Login
- Product search
- Cart
- Payment
- Order confirmation

All features together are tested as one complete system.

Purpose

- Ensure the full system works correctly
- Verify business requirements

8. Explain Acceptance Testing.

Acceptance Testing

Acceptance testing is the final level of testing performed to determine whether the software is ready for delivery and accepted by the customer or user.

Objectives

- Verify software meets business requirements
- Ensure user satisfaction
- Confirm software is ready for deployment

Types

- Alpha Testing
- Beta Testing

Example

A client checks whether a payroll system correctly calculates employee salaries before accepting the software.

Importance

- Builds customer confidence
- Ensures software fulfills user needs
- Reduces chances of failure after release

9. Differentiate Manual and Automation Testing.

Manual Testing

Testing is performed manually by humans

No programming knowledge required

Time-consuming

Suitable for small projects

More chances of human error

Low initial cost

Automation Testing

Testing is performed using tools/scripts

Requires programming knowledge

Faster execution

Suitable for large and repetitive projects

Less chances of human error

High initial setup cost

Manual Testing

Automation Testing

Example: Tester manually checks login page Example: Selenium automatically runs test cases

Conclusion

Manual testing is useful for exploratory and small-scale testing, while automation testing is useful for repetitive and large-scale testing.

10. Explain Functional and Non-functional Testing.

Functional Testing

Functional testing checks whether the software functions according to specified requirements.

Features

- Tests features and functionality
- Based on user requirements
- Focuses on inputs and outputs

Examples

- Login functionality
- Payment processing
- Registration form

Non-functional Testing

Non-functional testing checks the performance and quality attributes of software.

Features

- Tests system behavior
- Focuses on speed, security, usability, reliability, etc.

Examples

- Performance Testing
- Security Testing
- Load Testing

Difference

Functional Testing	Non-functional Testing
Checks what the system does	Checks how the system performs
Based on functional requirements	Based on quality attributes
Example: Login test	Example: Speed test

11. Explain Black-box Testing.

Black-box Testing

Black-box testing is a testing method in which the tester checks the functionality of software without knowing the internal code or program structure.

Features

- Focuses on inputs and outputs
- No programming knowledge required
- Based on requirements and specifications

Example

Testing a login page by entering username and password without knowing the internal code.

Advantages

- User-oriented testing
- Easy to perform

Disadvantages

- Internal code errors may remain undetected
-

12. Explain White-box Testing.

White-box Testing

White-box testing is a testing technique where the tester has knowledge of the internal code, logic, and structure of the program.

Features

- Tests internal working of software
- Requires programming knowledge
- Covers code paths, loops, and conditions

Example

Checking whether all conditions and loops inside a Java function execute correctly.

Advantages

- Improves code quality
- Detects hidden errors

Disadvantages

- Time-consuming
 - Requires skilled testers
-

13. Explain Grey-box Testing.

Grey-box Testing

Grey-box testing is a combination of black-box and white-box testing where the tester has partial knowledge of the internal structure of the application.

Features

- Tests both functionality and internal behavior
- Limited access to code/design
- Useful for web applications

Example

A tester knows database structure but tests the application through the user interface.

Advantages

- Better test coverage
- Finds defects effectively

Disadvantages

- Limited access to source code may miss some defects
-

14. Difference between Black-box and White-box Testing.

Black-box Testing

Tests software functionality

No knowledge of code required

Focuses on input and output

Performed by testers

Based on requirements

Finds missing functionality

White-box Testing

Tests internal code and logic

Knowledge of code required

Focuses on program structure

Usually performed by developers

Based on code design

Finds coding and logical errors

Example

- **Black-box:** Testing login page with valid/invalid inputs.
 - **White-box:** Checking internal conditions and loops in login code.
-

15. Explain Boundary Value Analysis with example.

Boundary Value Analysis (BVA)

Boundary Value Analysis is a testing technique in which testing is done on boundary values because errors commonly occur at boundaries.

It checks:

- Minimum value
- Maximum value
- Just below minimum

- Just above maximum

Example

Suppose a system accepts age between **18 to 60**.

Test cases:

Test Value Result

17	Invalid
18	Valid
19	Valid
59	Valid
60	Valid
61	Invalid

Here, boundary values are 18 and 60.

16. Explain Equivalence Partitioning with example.

Equivalence Partitioning

Equivalence Partitioning is a testing technique where input data is divided into valid and invalid groups called partitions. One test value from each partition is selected.

Example

Suppose marks allowed are from **0 to 100**.

Partitions:

- Invalid: less than 0
- Valid: 0 to 100
- Invalid: greater than 100

Test cases:

Input Partition

-5	Invalid
50	Valid
120	Invalid

Instead of testing every value, one value from each group is tested.

Advantages

- Reduces number of test cases
- Saves testing time

17. How to write an effective test case?

Steps to Write an Effective Test Case

1. Understand the requirement clearly.
2. Write a unique Test Case ID.
3. Provide a clear test case description.
4. Mention preconditions if any.
5. Specify test steps properly.
6. Define input test data.
7. Mention expected result clearly.
8. Keep the test case simple and understandable.
9. Ensure reusability and accuracy.

Example Format

Field	Example
Test Case ID	TC_01
Test Scenario	Login Functionality
Steps	Enter username and password

Field	Example
Test Data	admin / 1234
Expected Result	Login successful

18. What are the components of a test plan?

A test plan is a document that describes the complete testing process.

Components of a Test Plan

1. **Test Plan ID**
Unique identification of the test plan.
2. **Objective**
Purpose of testing.
3. **Scope**
Features to be tested and not tested.
4. **Test Strategy**
Testing approach and methods.
5. **Test Environment**
Hardware, software, and network setup.
6. **Resources and Responsibilities**
Team members and assigned tasks.
7. **Test Schedule**
Timeline and deadlines.
8. **Test Cases**
List of test cases to execute.
9. **Entry and Exit Criteria**
Conditions to start and stop testing.
10. **Risk and Mitigation**
Possible risks and their solutions.
11. **Deliverables**
Documents and reports produced during testing.

1. Explain Software Testing in detail and why it is important.

Software Testing

Software testing is the process of evaluating and verifying a software application to ensure that it works correctly according to specified requirements and is free from defects.

The main objective of software testing is to identify errors, improve quality, and ensure customer satisfaction before the software is released.

Testing can be performed manually or using automation tools.

Objectives of Software Testing

- Find defects and bugs
- Verify functionality
- Validate user requirements
- Improve software quality
- Ensure reliability and security
- Reduce future maintenance cost

Types of Software Testing

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing
- Functional Testing

- Non-functional Testing

Importance of Software Testing

1. **Improves Quality**
Ensures the software works correctly and efficiently.
2. **Detects Errors Early**
Bugs can be fixed before software release.
3. **Ensures Security**
Protects applications from vulnerabilities and attacks.
4. **Increases Customer Satisfaction**
Reliable software improves user experience.
5. **Reduces Development Cost**
Early bug fixing is cheaper than fixing after release.
6. **Ensures Performance**
Checks speed, stability, and reliability of software.

Conclusion

Software testing is an essential activity in software development that helps deliver high-quality, secure, and reliable software products.

2. Explain Verification vs Validation with examples.

Verification

Verification is the process of checking whether the software is being developed correctly according to specifications and design documents.

Key Points

- Process-oriented
- Done during development
- Does not execute code
- Prevents defects

Examples

- Requirement review
- Design inspection
- Code review

Question

“Are we building the product right?”

Validation

Validation is the process of checking whether the final software satisfies user requirements and business needs.

Key Points

- Product-oriented
- Performed after development
- Executes the software
- Detects defects

Examples

- Functional testing
- System testing
- User acceptance testing

Question

“Are we building the right product?”

Difference between Verification and Validation

Verification

Checks development process

Validation

Checks final product

Verification

Done before execution

No code execution required

Finds defects early

Example: Reviews

Validation

Done after execution

Requires code execution

Ensures user satisfaction

Example: Testing

3. Explain STLC (Software Testing Life Cycle) with diagram.

STLC (Software Testing Life Cycle)

STLC is a sequence of activities performed during the software testing process to ensure software quality.

Phases of STLC

1. Requirement Analysis

Testing team studies requirements and identifies testable aspects.

2. Test Planning

Test manager prepares:

- Test strategy
- Budget
- Resources
- Schedule

3. Test Case Development

Test cases and test data are created.

4. Test Environment Setup

Hardware and software environment is prepared for testing.

5. Test Execution

Test cases are executed and defects are identified.

6. Defect Reporting and Tracking

Bugs are reported to developers and tracked until fixed.

7. Test Closure

Testing activities are completed and reports are prepared.

STLC Diagram

Requirement Analysis



Test Planning



Test Case Development



Environment Setup



Test Execution



Defect Reporting & Tracking



Test Closure

Advantages of STLC

- Improves testing quality
- Reduces defects
- Provides systematic testing process
- Ensures timely delivery

4. Explain Qualification process in software testing.

Qualification in Software Testing

Qualification is the process of verifying and validating that software meets predefined standards, requirements, and intended use before release.

It ensures that the software is reliable, safe, and performs correctly.

Objectives of Qualification

- Ensure software quality
- Verify compliance with requirements
- Reduce risk and failures
- Increase customer confidence

Types of Qualification

1. Installation Qualification (IQ)

Checks whether software is installed correctly in the required environment.

Example

Verifying correct installation of a banking application on a server.

2. Operational Qualification (OQ)

Checks whether the software operates correctly under expected conditions.

Example

Testing login, data entry, and report generation functions.

3. Performance Qualification (PQ)

Checks whether the software performs effectively in real working conditions.

Example

Testing whether an e-commerce website handles thousands of users properly.

Qualification Process Steps

1. Requirement identification
2. Planning qualification activities
3. Executing tests
4. Recording results
5. Reporting defects
6. Final approval and release

Importance of Qualification

- Ensures software reliability
- Meets industry standards
- Improves product quality
- Reduces operational risks

5. Explain all levels of testing:

- o Unit
- o Integration
- o System
- o Acceptance

Software testing is divided into different levels to identify defects at different stages of development.

1. Unit Testing

Definition

Unit testing is the process of testing individual modules or components of a program separately to verify that each unit works correctly.

It is usually performed by developers.

Features

- Tests smallest unit of software
- Done during coding phase

- Detects coding errors early

Example

Testing a calculator's addition function separately.

```
int add(int a, int b){  
    return a + b;  
}
```

Testing:

- $\text{add}(2,3) = 5$
- $\text{add}(10,5) = 15$

Advantages

- Easy debugging
- Improves code quality
- Reduces development cost

2. Integration Testing

Definition

Integration testing checks whether multiple modules work correctly together after unit testing.

Features

- Tests communication between modules
- Detects interface errors
- Performed after unit testing

Example

In an e-commerce application:

- Login module
- Cart module
- Payment module

Integration testing checks whether these modules interact properly.

Types

- Top-down Testing
- Bottom-up Testing
- Big Bang Testing

Advantages

- Ensures smooth data flow
- Finds interaction defects

3. System Testing

Definition

System testing is the process of testing the complete integrated software system as a whole.

Features

- Tests entire application
- Performed in testing environment
- Includes functional and non-functional testing

Example

Testing a complete banking application including:

- Login
- Account management
- Money transfer
- Report generation

Advantages

- Verifies complete system behavior
- Ensures requirements are satisfied

4. Acceptance Testing

Definition

Acceptance testing is the final level of testing performed to determine whether the software is ready for delivery and accepted by the customer.

Features

- Performed by users or clients
- Checks business requirements
- Final testing stage

Types

Alpha Testing

Performed at developer site.

Beta Testing

Performed by real users in real environment.

Example

A company tests payroll software before approving and purchasing it.

Advantages

- Increases customer confidence
 - Ensures user satisfaction
-

6. Explain Manual Testing and Automation Testing in detail.

Manual Testing

Definition

Manual testing is a testing method in which testers execute test cases manually without using automation tools.

Features

- Human involvement required
- Suitable for small projects
- Useful for exploratory testing

Process

1. Read requirements
2. Create test cases
3. Execute test cases manually
4. Compare actual and expected results
5. Report defects

Advantages

- No programming knowledge required
- Flexible testing
- Better user experience checking

Disadvantages

- Time-consuming
- More human errors
- Difficult for repetitive tasks

Example

Tester manually checks login functionality by entering username and password.

Automation Testing

Definition

Automation testing uses tools and scripts to execute test cases automatically.

Features

- Fast execution
- Reusable test scripts
- Suitable for large projects

Common Tools

- Selenium
- JUnit
- TestNG
- QTP

Advantages

- Saves time
- Reduces human error
- Supports repeated execution

Disadvantages

- High initial cost
- Requires programming skills
- Maintenance of scripts needed

Example

Using Selenium to automatically test website login functionality.

Difference between Manual and Automation Testing

Manual Testing	Automation Testing
Performed manually	Performed using tools
Slower execution	Faster execution
More human errors	Fewer human errors
No coding required	Coding required
Best for small projects	Best for large projects

7. Explain Functional vs Non-functional Testing with examples.

Functional Testing

Definition

Functional testing checks whether software features work according to specified requirements.

Focus

- Inputs and outputs
- Functional requirements
- Business logic

Examples

- Login testing
- Registration form testing
- Payment processing testing

Types

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

Non-functional Testing

Definition

Non-functional testing checks performance and quality attributes of software.

Focus

- Speed
- Security
- Reliability
- Scalability
- Usability

Examples

- Load Testing
- Performance Testing
- Security Testing
- Stress Testing

Difference between Functional and Non-functional Testing

Functional Testing

Checks what the system does

Based on functional requirements

Tests features

Example: Login test

Non-functional Testing

Checks how the system performs

Based on quality attributes

Tests performance and behavior

Example: Performance test

8. Explain Black-box, White-box and Grey-box testing in detail.

Black-box Testing

Definition

Black-box testing is a testing technique in which the tester checks software functionality without knowing the internal code structure.

Features

- No programming knowledge required
- Based on requirements
- Focuses on input and output

Example

Testing login page using valid and invalid credentials.

Advantages

- User-oriented testing
- Easy to perform

Disadvantages

- Internal code errors may not be detected

White-box Testing

Definition

White-box testing is a testing technique where the tester has complete knowledge of internal code and logic.

Features

- Tests internal program structure
- Requires coding knowledge
- Covers loops, conditions, and paths

Example

Checking all conditions inside a Java login function.

Advantages

- Improves code quality
- Detects hidden errors

Disadvantages

- Time-consuming
- Requires skilled testers

Grey-box Testing

Definition

Grey-box testing is a combination of black-box and white-box testing where the tester has partial knowledge of the internal system.

Features

- Partial access to code/design
- Tests both functionality and internal behavior
- Common in web applications

Example

Tester knows database structure but tests application through UI.

Advantages

- Better test coverage
- Effective defect detection

Disadvantages

- Limited internal access
- Complex testing approach

Difference between Black-box, White-box and Grey-box Testing

Black-box	White-box	Grey-box
No code knowledge	Full code knowledge	Partial code knowledge
Tests functionality	Tests internal logic	Tests both
User perspective	Developer perspective	Combined perspective
Easy to perform	Complex	Moderate complexity

9. Compare all three testing techniques.

Difference between Black-box, White-box and Grey-box Testing

Black-box Testing	White-box Testing	Grey-box Testing
Tester has no knowledge of internal code	Tester has full knowledge of code	Tester has partial knowledge of code
Focuses on functionality	Focuses on internal logic and structure	Focuses on both functionality and internal behavior
No programming knowledge required	Programming knowledge required	Partial programming knowledge required
Based on requirements and specifications	Based on source code and design	Based on both requirements and design
Performed mainly by testers	Usually performed by developers	Performed by both testers and developers
Easy to perform	Complex and time-consuming	Moderate complexity
Example: Login page testing	Example: Checking code paths	Example: Testing UI with database knowledge

Conclusion

- **Black-box Testing** checks software behavior.
- **White-box Testing** checks internal code structure.
- **Grey-box Testing** combines both approaches for better coverage.

10. Explain Boundary Value Analysis in detail with diagram/example.

Boundary Value Analysis (BVA)

Boundary Value Analysis is a software testing technique in which test cases are designed using boundary values because errors usually occur at the edges of input ranges.

Instead of testing every possible value, testing is focused on:

- Minimum value
- Maximum value
- Just below minimum

- Just above maximum

Example

Suppose a system accepts age between **18 and 60**.

Valid Range

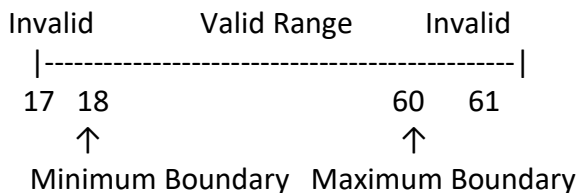
18 → 60

Test Cases

Test Value Result

17	Invalid
18	Valid
19	Valid
59	Valid
60	Valid
61	Invalid

Diagram



Advantages

- Detects boundary-related defects
- Reduces number of test cases
- Improves test effectiveness

Disadvantages

- Only useful for range-based inputs
- May miss defects inside the range

11. Explain Equivalence Partitioning with suitable example.

Equivalence Partitioning

Equivalence Partitioning is a testing technique in which input data is divided into groups called equivalence classes or partitions.

Each partition is expected to behave similarly, so only one value from each partition is tested.

Types of Partitions

1. Valid Partition
2. Invalid Partition

Example

Suppose marks allowed are from **0 to 100**.

Partitions

Partition	Type
Less than 0	Invalid
0 to 100	Valid
Greater than 100	Invalid

Test Cases

Input Value Result

Input Value Result

-5	Invalid
50	Valid
120	Invalid

Instead of testing all numbers from 0 to 100, one value from each partition is selected.

Advantages

- Reduces testing effort
- Saves time
- Avoids unnecessary test cases

Disadvantages

- Incorrect partitioning may miss defects

12. Explain how to write effective Test Plan and Test Cases.

Test Plan

Definition

A test plan is a document that describes the testing scope, strategy, objectives, schedule, and resources for a software project.

Components of Test Plan

1. Test Plan ID
2. Objective
3. Scope of Testing
4. Test Strategy
5. Test Environment
6. Resources and Responsibilities
7. Test Schedule
8. Entry and Exit Criteria
9. Risks and Solutions
10. Deliverables

Steps to Write Effective Test Plan

1. Understand project requirements
2. Define testing scope clearly
3. Select testing methods and tools
4. Assign responsibilities
5. Prepare realistic schedule
6. Define risk management plan
7. Review and update regularly

Test Case

Definition

A test case is a document containing inputs, execution steps, and expected results for testing a feature.

Components of Test Case

Field	Description
Test Case ID	Unique identification
Test Scenario	Functionality to test
Preconditions	Conditions before testing
Test Steps	Steps to execute

Field	Description
Test Data	Input values
Expected Result	Expected output
Actual Result	Actual output
Status	Pass/Fail

Steps to Write Effective Test Cases

1. Understand requirements clearly
2. Write simple and clear steps
3. Use proper test data
4. Define expected result accurately
5. Avoid duplicate test cases
6. Cover positive and negative scenarios
7. Keep test cases reusable

Example Test Case

Field	Example
Test Case ID	TC_01
Test Scenario	Login Function
Steps	Enter username and password
Test Data	admin / 1234
Expected Result	Login successful
Status	Pass/Fail

UNIT-5 Advanced Testing and Automation Testing Tool (Selenium)